INTRODUCTION TO BANKING

MIDTERM PROJECT REPORT

Instructor: Prof.Dr. Yüksel Akay ÜNVAN

Project Title: Digital Piggy Bank System

Developers: Elif Nur Aygün (22030411041)

Kardelen Dobrucalı (23030411023)

1. EXECUTIVE SUMMARY

Traditional savings accounts are often too passive and boring to motivate young customers to save money regularly. To solve this, we developed the "Digital Piggy Bank System." This is a goal-oriented savings prototype that uses simple gamification to make saving fun and engaging. Instead of just holding money like a standard bank account, our system acts as a smart financial friend who will help us save money. It asks for the user's monthly income and expenses to calculate a safe savings limit. Based on this data, it estimates exactly when the user will reach their goal, helping them save for their dreams without ruining their daily budget.

2. CONNECTION TO BANKING CLASSES

2.1. Week 4: Banking Products & Services (Generating Deposits)

Banks always need cheap funding, and customer deposits are the best way to get it. Our Digital Piggy Bank is a great tool for creating current savings account funds.

- **Small but Steady Deposits:** When customers save small amounts regularly for a specific goal, the bank gets a stable and low-cost pool of money.
- **Customer Loyalty:** If a customer is emotionally attached to reaching their goal on our app, they are much less likely to move their money to a competitor bank.

2.2. Week 10: Digital Banking (Personalized Experience)

Today, a banking app shouldn't just be a place to check your balance; it should be a helpful assistant.Some banks already have similar features, but we are offering something different. Instead of typical assistant, we are creating a financial companion enriched with the image of a piggy bank, acting like a friend with whom customers can connect.

- **Gamification:** By adding a cute "Piggy" mascot, progress bars, and celebration messages, we turn saving money from a stressful task into a rewarding game.
- **Smart Planning:** Instead of just waiting for the user to deposit money, our system calculates their "Safe Savings Power" based on their income and expenses. It then tells them exactly how many months it will take to reach their goal.

3. HOW THE PROTOTYPE WORKS

We built this prototype in Python. It runs in the terminal but uses colors, shapes, and clear menus to feel like a real digital banking app.

3.1. Getting Started

When the user opens the app, we don't just ask for numbers. We ask, "What is your dream?" to make it personal. Then, we ask for their monthly income and mandatory expenses to understand their real financial situation.

On the board:

```python
while True:
    try:
        monthly_income = float(input(f"{O}STEP 3: For me to draw a complete strategy, what is your monthly net income?: "))
        if monthly_income <= 0:
            print(f"{O}{RED}❌ Income must be greater than 0.{RESET}")
            continue
        break
    except ValueError:
        print(f"{O}{RED}❌ ERROR: Please enter numbers only!{RESET}")

while True:
    try:
        monthly_expense = float(input(f"{O}STEP 4: Lastly, what are your total mandatory monthly expenses?: "))
        if monthly_expense < 0:
            print(f"{O}{RED}❌ Expenses cannot be negative.{RESET}")
            continue
        break
    except ValueError:
        print(f"{O}{RED}❌ ERROR: Please enter numbers only!{RESET}")

print(f"\n{O}{GREEN}✨ Supersonic! Data processed. Piggy is ready for you!{RESET}")
time.sleep(1.5)

piggy_bank = PersonalFinanceAssistant(goal_name, target_amount, monthly_income, monthly_expense)
```

```python
        if percentage >= 100:
            print(CUP_LOGO)
        else:
            print(PIGGY_LOGO)

        print(O + "★"*65)
        print(f"{O}💳 Monthly Net Income : {GREEN}{self.monthly_income:,.2f} TL{RESET}")
        print(f"{O}💸 Monthly Expenses   : {RED}{self.monthly_expense:,.2f} TL{RESET}")
        print(f"{O}🛡️  Savings Power      : {CYAN}{self.savings_capacity:,.2f} TL / Month{RESET}")
        print(O + "-" * 65)
        print(f"{O}🎯 Goal Name          : {BOLD}{self.goal_name}{RESET}")
        print(f"{O}💰 Target Amount      : {self.target_amount:,.2f} TL")
        print(f"{O}💵 Money in Piggy     : {GREEN}{self.current_balance:,.2f} TL{RESET}")
        print(f"{O}📉 Remaining Amount   : {YELLOW}{remaining_amount:,.2f} TL{RESET}")
        print("\n" + O + "  " + self.draw_progress_bar())
        print(O + "★"*65)
```

```python
try:
    print("\n" + O + "#"*65)
    print(PIGGY_LOGO)
    print(O + "#"*65)
    print(f"\n{O}Hi! I am Piggy, your personal digital piggy bank.")
    print(f"{O}Tell me your goal, and let's handle the rest together!\n")

    goal_name = input(f"{O}STEP 1: What is that incredibly exciting goal? (e.g., Motorcycle, Vacation): ")

    while True:
        try:
            target_amount = float(input(f"{O}STEP 2: Oh, great idea! How much total money do we need for '{goal_name}'?: "))
            if target_amount <= 0:
                print(f"{O}{RED}❌ Amount must be greater than 0.{RESET}")
                continue
            break
        except ValueError:
            print(f"{O}{RED}❌ ERROR: Please enter numbers only!{RESET}")
```

On the prototype:

```
#################################################################
            ✨  🟡   ✨
               \
          _.-_ _...._ .-_
         /    _O     O \        🎛 DIGITAL PIGGY BANK 🎛
         |   ( o o )   |
         \    '---'    /         "Your sweetest companion
          '._         _'          on the path to your dreams!"
            /'-----'\
           //       \\

#################################################################

Hi! I am Piggy, your personal digital piggy bank.
Tell me your goal, and let's handle the rest together!

STEP 1: What is that incredibly exciting goal? (e.g., Motorcycle, Vacation): Phone
STEP 2: Oh, great idea! How much total money do we need for 'Phone'?: 100000
STEP 3: For me to draw a complete strategy, what is your monthly net income?: 35000
STEP 4: Lastly, what are your total mandatory monthly expenses?: 20000

✨ Supersonic! Data processed. Piggy is ready for you!

----------------------------------------
         PIGGY CONTROL CENTER
----------------------------------------
1 ➔ 📊 Piggy Dashboard
2 ➔ 💸 Deposit Money
3 ➔ 💡 Piggy's Future Prediction
0 ➔ 🚪 Secure Exit
----------------------------------------
What would you like to do? (0/1/2/3): |
```

3.2. Piggy Control Center and Dashboard

After the setup, the user sees the main dashboard. Instead of a boring bank statement, they see our Piggy logo, a progress bar, how much money they have, and how much is left to reach the target.

On the board:

```python
class PersonalFinanceAssistant:
    def __init__(self, goal_name, target_amount, monthly_income, monthly_expense):
        self.goal_name = goal_name
        self.target_amount = target_amount
        self.monthly_income = monthly_income
        self.monthly_expense = monthly_expense
        self.current_balance = 0.0
        self.deposit_history = []
        self.savings_capacity = max(0, self.monthly_income - self.monthly_expense)

    def send_motivation(self):
        quote = random.choice(MOTIVATIONAL_QUOTES)
        print(f"\n{O}{PURPLE}✨ PIGGY SAYS: \"{quote}\"{RESET}")
```

```python
while True:
    print("\n" + O + "-"*40)
    print(f"{O}          {BOLD}PIGGY CONTROL CENTER{RESET}")
    print(O + "-"*40)
    print(f"{O}{PINK}1{RESET} ➔ 📊 Piggy Dashboard")
    print(f"{O}{PINK}2{RESET} ➔ 💸 Deposit Money")
    print(f"{O}{PINK}3{RESET} ➔ 💡 Piggy's Future Prediction")
    print(f"{O}{PINK}0{RESET} ➔ 🚪 Secure Exit")
    print(O + "-"*40)

    choice = input(f"{O}What would you like to do? (0/1/2/3): ")

    if choice == '1':
        piggy_bank.show_dashboard()
```

On the prototype:

```
✨ Supersonic! Data processed. Piggy is ready for you!

----------------------------------------
         PIGGY CONTROL CENTER
----------------------------------------
1 → 📊 Piggy Dashboard
2 → 💸 Deposit Money
3 → 💡 Piggy's Future Prediction
0 → 🚪 Secure Exit
----------------------------------------
What would you like to do? (0/1/2/3): 1

★★★
```

3.3. Deposit Money and Warnings

When you select option number 2, "deposit money," the system allows you to deposit funds, enabling you to track your progress. However, the smart assistant Piggy is designed to send a warning if it detects that you have exceeded the secure monthly deposit limit. This ensures the system is user-friendly and budget friendly.

On the board:

```
elif choice == '2':
    print(f"\n{O}--- DEPOSIT TRANSACTION ---")
    try:
        amount = float(input(f"{O}How much are you putting in Piggy today? (TL): "))
        if amount > 0:
            piggy_bank.deposit_money(amount)
        else:
            print(f"{O}{RED}❌ ERROR: Please enter an amount greater than 0.{RESET}")
    except ValueError:
        print(f"{O}{RED}❌ ERROR: Please enter numbers only!{RESET}")
```

```
def deposit_money(self, amount):
    old_percentage = (self.current_balance / self.target_amount) * 100 if self.target_amount > 0 else 0
    self.current_balance += amount
    self.deposit_history.append(amount)
    new_percentage = (self.current_balance / self.target_amount) * 100 if self.target_amount > 0 else 0

    print(f"\n{O}{YELLOW}💰 Shaking the piggy bank and dropping money...{RESET}")
    time.sleep(1)
    print(f"{O}{GREEN}✅ TRANSACTION SUCCESSFUL! {amount:,.2f} TL added to your piggy bank.{RESET}")

    if new_percentage >= 100 and old_percentage < 100:
        print(f"\n{CUP_LOGO}")
    elif new_percentage >= 75 and old_percentage < 75:
        print(f"\n{O}{CYAN}🔥 AWESOME! You completed 75% of the way. Very close!{RESET}")
    elif new_percentage >= 50 and old_percentage < 50:
        print(f"\n{O}{CYAN}🌟 HALFWAY THERE! You are doing great.{RESET}")
    elif new_percentage >= 25 and old_percentage < 25:
        print(f"\n{O}{CYAN}👍 FIRST QUARTER DONE! That's true determination.{RESET}")

    if amount > self.savings_capacity:
        print(f"\n{O}{YELLOW}⚠️ Piggy Warns: You exceeded your safe savings capacity.")
        print(f"{O}Going fast is good, but please don't shake up your budget!{RESET}")

    time.sleep(1)
    self.send_motivation()
```

On the prototype:

```
----------------------------------------
          PIGGY CONTROL CENTER
----------------------------------------
1 → 📊 Piggy Dashboard
2 → 💸 Deposit Money
3 → 💡 Piggy's Future Prediction
0 → 🚪 Secure Exit
----------------------------------------
What would you like to do? (0/1/2/3): 2

--- DEPOSIT TRANSACTION ---
How much are you putting in Piggy today? (TL): 15000

💰 Shaking the piggy bank and dropping money...
✅ TRANSACTION SUCCESSFUL! 15,000.00 TL added to your piggy bank.

✨ PIGGY SAYS: "Don't give up! You are closer to your dreams than you think."

----------------------------------------
          PIGGY CONTROL CENTER
----------------------------------------
1 → 📊 Piggy Dashboard
2 → 💸 Deposit Money
3 → 💡 Piggy's Future Prediction
0 → 🚪 Secure Exit
----------------------------------------
What would you like to do? (0/1/2/3): 2

--- DEPOSIT TRANSACTION ---
How much are you putting in Piggy today? (TL): 20000

💰 Shaking the piggy bank and dropping money...
✅ TRANSACTION SUCCESSFUL! 20,000.00 TL added to your piggy bank.

👍 FIRST QUARTER DONE! That's true determination.

⚠️ Piggy Warns: You exceeded your safe savings capacity.
Going fast is good, but please don't shake up your budget!

✨ PIGGY SAYS: "Future you will thank present you!"
```

As you saw in this prototype example, we coded it to react differently when the suggested budget is loaded and when the suggested budget is exceeded, and to provide warnings to help you avoid disrupting your budget.

3.4. Piggy's Future Prediction

In the future prediction section, the intelligent assistant Piggy compares the amount you have deposited with your ultimate goal and calculates the time required to reach that goal.

On the board:

```
elif choice == '3':
    piggy_bank.run_financial_analysis()
```

```
def run_financial_analysis(self):
    print("\n" + O + "="*65)
    print(f"{O}{YELLOW}💡 PIGGY AI FINANCIAL COACH 💡{RESET}")
    print(O + "="*65)
    remaining_amount = max(0, self.target_amount - self.current_balance)

    if remaining_amount == 0:
        print(f"{O}{GREEN}Goal reached! Piggy is proud of you.{RESET}")
        return

    print(f"{O}🎯 Your Goal: {BOLD}{self.goal_name}{RESET}")
    print(f"{O}📉 Remaining Amount to Climb: {remaining_amount:,.2f} TL")

    if self.savings_capacity <= 0:
        print(f"\n{O}{RED}⚠️ Piggy is Sad: Your expenses are equal to or greater than your income.")
        print(f"{O}Please create a small savings space by cutting your expenses first.{RESET}")
    else:
        estimated_months = math.ceil(remaining_amount / self.savings_capacity)
        print(f"{O}💼 Your Monthly Safe Savings Power: {self.savings_capacity:,.2f} TL")
        print(f"\n{O}{CYAN}👉 PIGGY'S PREDICTION:{RESET} If you continue with this discipline,")
        print(f"{O}in exactly {BOLD}{GREEN}{estimated_months} months{RESET}, '{self.goal_name}' will be yours!")

    print(O + "="*65)
    self.send_motivation()
```

On the prototype:

```
----------------------------------------
         PIGGY CONTROL CENTER
----------------------------------------
1 → 📊 Piggy Dashboard
2 → 💸 Deposit Money
3 → 💡 Piggy's Future Prediction
0 → 🚪 Secure Exit
----------------------------------------
What would you like to do? (0/1/2/3): 3

==================================================================
💡 PIGGY AI FINANCIAL COACH 💡
==================================================================
🎯 Your Goal: Phone
📉 Remaining Amount to Climb: 65,000.00 TL
💼 Your Monthly Safe Savings Power: 15,000.00 TL

👉 PIGGY'S PREDICTION: If you continue with this discipline,
in exactly 5 months, 'Phone' will be yours!
==================================================================

✨ PIGGY SAYS: "Don't give up! You are closer to your dreams than you think."
```

3.5. The Future Prediction Bar and Achieving the Goal

The future prediction bar shows you the time remaining based on the amount you've loaded; this indicator provides a more concrete view of your goal. If you deposit the safe amount into your Piggy account each month, you will reach your goal within the timeframe predicted by the system. For a more visual representation, Piggy will congratulate you with a visual trophy once you achieve this timeframe.

On the board:

```python
def draw_progress_bar(self):
    if self.target_amount == 0: return "[                    ] 0.00%"
    percentage = (self.current_balance / self.target_amount) * 100
    if percentage > 100: percentage = 100

    filled_part = int(20 * percentage // 100)
    empty_part = 20 - filled_part
    bar = f"{GREEN}{'▓' * filled_part}{RESET}{'▒' * empty_part}"

    return f"[{bar}] {percentage:.2f}%"
```

```python
def show_dashboard(self):
    remaining_amount = max(0, self.target_amount - self.current_balance)
    percentage = (self.current_balance / self.target_amount) * 100

    print("\n" + O + "★"*65)

    if percentage >= 100:
        print(CUP_LOGO)
    else:
        print(PIGGY_LOGO)
```

```python
CUP_LOGO = f"""
{O}       {YELLOW} .-========-. {RESET}
{O}       {YELLOW} \\'-======-'/ {RESET}   {GREEN}🎉 INCREDIBLE SUCCESS! 🎉{RESET}
{O}       {YELLOW} _|   .=.   |_ {RESET}   You reached your goal. Time to
{O}       {YELLOW}((|  {{1}}  |)) {RESET}   turn your dreams into reality!
{O}       {YELLOW} \\|   /|\\   |/ {RESET}
{O}       {YELLOW}  \\__ '`' __/  {RESET}
{O}       {YELLOW}    _`) (`_    {RESET}
{O}       {YELLOW}  _/_______\\_  {RESET}
{O}       {YELLOW} /___________\\ {RESET}
"""
```

On the prototype:

```
★★★
```

4. WHY BANKS WOULD LOVE THIS BUSINESS MODEL

1. **Cross-Selling Opportunities:** If a user is at 90% of their goal but needs the item right now (like a laptop), the bank knows exactly what they are saving for. The bank can offer a small, customized personal loan or credit card installments to help them finish the goal immediately.

2. **Valuable Customer Data:** When users type in their goals (like "vacation," "motorcycle," or "wedding"), the bank learns exactly what its customers want to buy. If the bank sees that thousands of young users are saving for travel, they can create special travel credit card campaigns just for them.

3. **Cheap Deposits for the Bank:** Banks need money to give out loans. When thousands of users save small amounts every week in these digital piggy banks, it creates a massive pool of cheap funds for the bank. Since these goal-based accounts usually pay little to no interest to the user, the bank gets a very low-cost funding source.

4. **Brand Partnerships (E-commerce):** Because the bank knows what the user is saving for, they can make deals with big brands. For example, if a user reaches their target to buy a new smartphone, the bank can offer a special 5% discount code if they buy it using the bank's debit card.

5. **Attracting Young Customers:** Traditional banking is boring for students and young professionals. A fun, game-like app with a cute mascot makes the bank look modern and friendly. It brings younger generations into the banking system early, turning them into lifelong customers.

6. **Teaching Financial Responsibility:** By warning users who try to save more than they can afford, the bank shows it cares about the customer's real financial health, not just taking their money. This builds huge trust and loyalty between the customer and the bank.

5. CONCLUSION

We started this project to solve a simple problem: The difficulty of saving money and the lack of motivation felt in this regard, especially by young people. The Digital Piggy Bank completely changes this dynamic by turning a basic banking product into a fun, interactive, and personalized financial coach.

Through our Python prototype, we demonstrated how digital banking can go beyond just showing an account balance. By analyzing a user's income and mandatory expenses, our system calculates a "safe savings capacity" and predicts their success. This proves that a banking app can actively protect a customer's daily budget while still helping them reach their biggest dreams. The addition of the Piggy mascot, visual progress bars, and motivational quotes shows how simple gamification can build strong, lifelong financial habits.

Ultimately, this project perfectly ties together what we learned in our Introduction to Banking classes. It uses the modern, user-friendly digital experiences discussed in Week 10 to generate the low-cost, stable customer deposits and cross-selling opportunities needed in Week 4.

By doing so, the Digital Piggy Bank creates a true win-win situation. The customer gets a smart, caring, and engaging tool to achieve their goals safely, and the bank gets a highly loyal, data-rich, and profitable customer base for the future.